

# 실행 환경

각 트랙이 스타터 킷과 함께 배포하는 실행 환경 계약을 한 곳에 모았습니다. 채점 머신은 여기 적힌 인터프리터와 패키지로 제출물을 실행합니다.

## Track 1 · agent.py 실행 환경

### Track 1 — 채점 런타임

`usvnav docs` 가 `usvnav 0.1.0`의 코드에서 생성한 문서의 한국어판입니다.

제출물이 채점 시점에 만나는 환경입니다. 식별자는 `usvnav-runtime/1`이며, 매니페스트의 `runtime` 필드에 이 값을 적어야 합니다.

| 항목                           | 값                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python                       | 3.13 (major.minor 고정, 패치 버전은 고정하지 않음)                                                                                                                                                                                                                                                                                                                         |
| <code>numpy</code>           | 2.5.3, 정확히 고정                                                                                                                                                                                                                                                                                                                                                 |
| <code>onnxruntime-gpu</code> | 1.23.2, 정확히 고정                                                                                                                                                                                                                                                                                                                                                |
| CUDA 라이브러리                   | onnxruntime-gpu가 불러오는 pip 패키지들, 정확히 고정: <code>nvidia-cuda-runtime-cu12=12.9.79</code> , <code>nvidia-cudnn-cu12=9.25.1.1</code> , <code>nvidia-cublas-cu12=12.9.2.10</code> , <code>nvidia-cufft-cu12=11.4.1.4</code> , <code>nvidia-curand-cu12=10.3.10.19</code> , <code>nvidia-cuda-nvrtc-cu12=12.9.86</code> , <code>nvidia-nvjitlink-cu12=12.9.86</code> |
| execution providers          | <code>CUDAExecutionProvider</code> , <code>CPUExecutionProvider</code>                                                                                                                                                                                                                                                                                        |
|                              |                                                                                                                                                                                                                                                                                                                                                               |

| 항목        | 값                                                                                                                                                                                                                     |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 그 밖의 패키지  | 런타임 자체의 패키지 — 표준 라이브러리, numpy, onnxruntime, <code>usvnav</code> — 는 import할 수 있습니다. 그 밖에 에이전트가 import하는 것은 모두 제출 디렉터리 안에 순수 파이썬으로, 또는 Linux x86_64 / CPython 3.13용으로 빌드된 wheel(컴파일된 확장 포함)을 그 안에 풀어 둔 형태로 들어 있어야 합니다  |
| 네트워크      | 없음                                                                                                                                                                                                                    |
| CPU       | Intel Core i7-14700 (28스레드)                                                                                                                                                                                           |
| RAM       | 30 GB                                                                                                                                                                                                                 |
| GPU       | NVIDIA GeForce RTX 5060 (8 GB). 제출물은 onnxruntime의 CUDAExecutionProvider로 GPU에 닿습니다                                                                                                                                    |
| 공유        | 채점 PC는 여러 에이전트(다른 에피소드나 다른 제출물)를 동시에 실행할 수 있으며, 이들은 CPU와 GPU를 함께 씁니다. 에이전트에 CPU 코어를 따로 배정하지 않으며, 시간 한도는 이런 조건에서 켜 벽시계 시간입니다                                                                                           |
| 메모리 제한    | 킷이 아니라 채점 샌드박스가 정합니다. 문제 설명을 보십시오                                                                                                                                                                                     |
| 시간 제한     | 에피소드마다 벽시계 120초 — <code>reset(meta)</code> 호출부터 에피소드가 끝날 때까지 에이전트와 채점기가 쓴 시간을 합쳐서 잽니다(넘으면 그 에피소드는 <code>time_overrun</code> 이 되고, 다음 에피소드를 위해 에이전트 프로세스를 다시 시작합니다). <code>Agent(config_path)</code> 는 120초 안에 끝나야 합니다 |
| 제출 형태     | 디렉터리                                                                                                                                                                                                                  |
| 에이전트 프로세스 | 러너의 자식 프로세스이며, 관측과 액션은 로컬 파이프로 주고받습니다. 조건마다 프로세스 하나가 한 번 생성되어 그 조건의 에피소드를 사이에 <code>reset()</code> 을 호출하며 차례로 실행합니다. 종료되었거나 에피소드의 시간 한도를 넘긴 프로세스는 다음 에피소드를 위해 다시 시작합니다                                                |
| 벽시계       | 실시간이 아닙니다 — 러너는 <code>act()</code> 를 기다립니다. 틱 하나에 대한 한도는 없고, 에피소드의 한도만 있습니다                                                                                                                                           |

인터프리터와 패키지 구성은 대회 기간에 바뀌지 않습니다. 배포한 `usvnav` 패키지는 numpy 외에 아무것도 의존하지 않습니다: 채점기가 실행하는 모든 것이 네트워크 없는 머신에 있어야 하기 때문입니다. 러너가 시간 한도를 적용하는 방식은 `docs/RULES.md` §4에 있습니다. 대회 규칙(한도, 채점, 순위, 공개 범위, 마감)은 사이트의 문제 설명과 공통 규정(<https://kraftonai-autonomy-hackathon.com/problems/>)이 기준입니다.

# onnxruntime

런타임은 GPU 빌드 `onnxruntime` 을 제공하므로, 학습된 구성 요소를 프레임워크와 무관하게 실행하는 방법은 하나입니다: PyTorch, TensorFlow, JAX에서 ONNX로 내보내고, `.onnx` 파일을 제출물 안에 넣은 뒤 `onnxruntime.InferenceSession(path, providers=["CUDAExecutionProvider", "CPUExecutionProvider"])` 로 엽니다. Track 3가 고정한 것과 같은 버전이 채점 머신에 한 번 설치되어 있으며, 그 외에 설정한 것은 없습니다(기본 세션 옵션, TensorRT 없음). CUDA provider는 복사 비용을 상쇄할 만큼 큰 모델에서만 확실히 이득이고, 작은 네트워크는 CPU provider가 더 빠른 경우가 많습니다. 어느 쪽이든 틱당 벽시계 시간은 시간 제한에 들어갑니다.

로컬에서 같은 환경을 맞추려면 onnxruntime만이 아니라 인터프리터와 numpy도 맞춰야 합니다. 킷은 Python 3.10 이상, numpy 1.24 이상이면 어디에나 설치되지만 채점기는 Python 3.13과 numpy 2.5.3으로 돌고, numpy 1.x와 2.x는 타입 승격과 복사 의미가 다르므로 채점기의 버전으로 개발하십시오:

명령

```
python --version
```

```
pip install numpy==2.5.3
```

```
pip install onnxruntime-gpu==1.23.2 nvidia-cuda-runtime-cu12==12.9.79 nvidia-cudnn-cu12==9.25.1
```

```
pip install onnxruntime==1.23.2
```

GPU 빌드는 CUDA 라이브러리를 위 pip 패키지에서 찾습니다. 시스템 CUDA 설치 필요하지 않고 사용하지도 않습니다.

원문(영문): 스타터 킷의 `RUNTIME.md`. 이 페이지는 그 한국어판이며, 값이 다르면 원문이 기준입니다.

# Track 2 · 2-1, 2-2 (하나의 킷)

## Track 2 — 채점 런타임

아래 내용 전부가 제출물이 실행되는 환경이자 제출물을 실행하는 것들입니다. 채점 PC 한 대는 한 번에 제출물 하나를 채점합니다.

## 인터프리터와 패키지

| 버전     |                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python | 3.10.12 (Ubuntu 22.04.5, Linux 6.8.0-138-generic)                                                                                                           |
| numpy  | 1.21.5                                                                                                                                                      |
| scipy  | 1.8.0                                                                                                                                                       |
| pillow | 12.1.0                                                                                                                                                      |
| torch  | 2.8.0+cu128 (CUDA 12.8), torchvision 0.23.0+cu128과 함께 — 주최 측의 2-2 학습기가 사용합니다. 킷의 <code>usvsim.guide.learner</code> 에는 torch만 필요합니다. 2-1 제출물에는 둘 다 제공되지 않습니다 |

2-1 제출물은 **numpy, scipy, pillow**와 **Python** 표준 라이브러리만 **import**할 수 있습니다. 자체 컴파일 확장 없음, 채점 시점의 패키지 설치 없음, 네트워크 없음. 채점은 numpy 1.21.5로 돌므로 그 버전으로 개발하고 시험하십시오.

## 하드웨어

|     |                                                                 |
|-----|-----------------------------------------------------------------|
| CPU | Intel Core i7-14700, 논리 코어 28개. 2-1 제출물은 <b>한 코어</b> 에 고정됩니다    |
| RAM | 30 GB. 2-1 제출물은 <b>8 GB</b> 상한, 2-2 채점은 <b>24 GB</b> 상한에서 실행됩니다 |
| GPU | NVIDIA GeForce RTX 5060 (8 GB). 주최 측의 2-2 학습기만 사용합니다            |

## 2-1 제출물이 실행되는 조건

- 한 코어에 고정된 자식 프로세스이며, 경로에는 킷만 있습니다(정답 환경의 plant는 import할 수 없습니다).
- 비공개 세트 전체 재생(360회, 각 120초)에 벽시계 600초. `usvsim speed-check` 가 제출물의 소요 시간을 외삽해 줍니다.
- CPU만, 네트워크 없음(비특권 네트워크 네임스페이스), 메모리 상한 8 GB.
- 시간 초과나 비정상 종료가 나면 실행이 끝납니다.
- 제출 시뮬레이터는 `make_sim` 에 고정된 `seed` 하나를 주어 한 번 만들고, 한 번 채점합니다.

## 2-2 제출물이 실행되는 조건

코드가 아니라 데이터를 제출합니다. 접수에서 디렉토리를 검증하고(`usvsim validate-dataset` 이 비공개 프레임 비교를 제외한 같은 검사를 보여 줍니다), 고정 학습기를 그 데이터로 학습시킨 뒤(`usvsim.guide.learner` 의 방식, 30 epoch, 고정 시드), 각 정책을 비공개 세트에서 롤아웃합니다. 채점 한 건에는 약 30분이 듭니다(이전 채점 장비 기준 실측, 새 장비에서 재측정 예정). 결함이 난 실행을 포함해 채점 방식은 대회 규칙이므로 사이트의 문제 설명과 공통 규정(<https://kraftonai-autonomy-hackathon.com/problems/>)을 따릅니다.

원문(영문): 스타터 킷의 `RUNTIME.md`. 이 페이지는 그 한국어판이며, 값이 다르면 원문이 기준입니다.

## Track 3 · 채점 컨테이너

### Track 3 — 채점 런타임

`wam/runtime.py` 와 `runtime/scorer-requirements.txt` 에서 2026-10-02에 생성한 문서의 한국어판입니다. 공식 채점기는 정확히 이것을 실행합니다.

### 그래프를 실행하는 것

- **ONNX Runtime 1.23.2** (`onnxruntime-gpu`), **CUDAExecutionProvider**, GPU 하나. 설치된 버전이 1.23.2가 아니거나 CUDA provider가 활성이 아니면 채점기는 시작을 거부합니다 — CPU로 조용히 넘어가는 일은 없습니다(`wam.runtime.make_session`).
- Provider 옵션은 대회 기간 동안 고정입니다:

| 옵션                     | 값 |
|------------------------|---|
| <code>device_id</code> | 0 |

| 옵션                                     | 값               |
|----------------------------------------|-----------------|
| <code>cudaconv_algo_search</code>      | HEURISTIC       |
| <code>use_tf32</code>                  | 0               |
| <code>do_copy_in_default_stream</code> | 1               |
| <code>arena_extend_strategy</code>     | kNextPowerOfTwo |

`cudaconv_algo_search = HEURISTIC` 은 시간 측정 기반 자동 튜닝을 하지 않는다는 뜻입니다(cuDNN 9의 `DEFAULT` 는 느린 대체 엔진을 고릅니다). `use_tf32 = 0` 은 CPU 경로와 같은 fp32 연산을 유지합니다. `device_id` 는 어느 카드인지를 가리키는 값이고 수치 설정이 아닙니다(GPU가 여러인 호스트에서는 `WAM_CUDA_DEVICE` 로 바꿉니다).

- 세션 옵션: `intra_op_num_threads` 4, `inter_op_num_threads` 1, 그래프 최적화 `ORT_ENABLE_ALL` .
- 배치 크기: 채점은 행을 최대 8개씩 묶어 실행하지만, **모든 한도는 B = 1에서** 에피소드 하나씩 측정합니다: 프레임당 생성 시간 p95(100 ms 이하), 제어 스텝 p95(40 ms 이하), 채점기 프로세스의 최대 GPU 메모리(6 GB 이하), `encode` 벽시계(1 s 이하). 그래프는 B가 무엇이든 행마다 같은 답을 내야 합니다.
- 결정성: 검증기가 각 그래프를 같은 임의 입력(`B = 2`)으로 두 번 실행해 **1e-4**(절대·상대) 안에서 같은 출력을 요구하고, 그 입력의 두 번째 행만 따로 실행한 출력이 배치의 두 번째 행과 같은 허용 오차 안에서 같아야 합니다. `Random*` 연산자는 그래프 검사에서 거부됩니다. 러너는 모든 블록의 시작 잠재값을 공개된 시드 번들(`contract.seed_bundle` , `initial_latent` )에서 공급합니다.
- 정밀도는 참가자의 선택입니다: 그래프 자체의 dtype으로 실행되며, 낮은 정밀도도 결정성 허용 오차는 지켜야 합니다.

## 실행 환경

채점기의 Python 환경(Python 3.10)은 아래 wheel 세트를 PyPI에서 설치한 것과 정확히 같습니다. CUDA 12.9와 cuDNN 9 런타임 라이브러리는 wheel로 들어오므로, 호스트에는 CUDA를 지원하는 NVIDIA 드라이버만 있으면 됩니다. wheel 세트와 위의 provider 옵션이 고정 내용의 수치 전부입니다. 컨테이너로 재현하려면 `nvidia/cuda:12.9.1-base-ubuntu22.04` 위에 Python 3.10으로 wheel 세트를 설치하십시오(GPU는 NVIDIA container toolkit으로 연결).

## 같은 것을 로컬에서 실행하기

- 검증: `python -m wam.validator <submission_dir>` — 접수가 실행하는 것과 같은 코드를 CPU provider에서 돌립니다(`onnxruntime` CPU wheel로 충분하며, 프레임은 CUDA 경로와 uint8 반올림 안에서 일치합니다).
- 채점기 환경 재현: CUDA 12 드라이버 위에서 `python -m pip install -r runtime/scorer-requirements.txt` 를 실행한 뒤 `python -m wam.runtime` 을 실행하면 고정 검사 결과 (`onnxruntime_matches_pin`, `cuda_available`)를 출력합니다.
- 채점 GPU는 NVIDIA GeForce RTX 5060(8 GB)이며, Intel Core i7-14700과 30 GB 메모리를 갖춘 PC에서 한 번에 제출물 하나씩 채점합니다. 시간 한도는 채점 때 이 카드에서 잤 값으로 판정합니다. 참가자 카드의 벽시계 수치는 채점기의 수치가 아닙니다. 파라미터 상한(0.5B) 근처의 큰 그래프는 이 카드에서 시간 한도를 넘을 수 있으므로, 크기 상한보다 시간 한도가 먼저 걸린다고 보고 설계하십시오.

## wheel 세트 ( `runtime/scorer-requirements.txt` )

| 패키지                      | 버전        |
|--------------------------|-----------|
| coloredlogs              | 15.0.1    |
| exceptiongroup           | 1.3.1     |
| flatbuffers              | 25.12.19  |
| humanfriendly            | 10.0      |
| iniconfig                | 2.3.0     |
| ml_dtypes                | 0.6.0     |
| mpmath                   | 1.3.0     |
| numpy                    | 2.2.6     |
| nvidia-cublas-cu12       | 12.9.2.10 |
| nvidia-cuda-nvrtc-cu12   | 12.9.86   |
| nvidia-cuda-runtime-cu12 | 12.9.79   |
| nvidia-cudnn-cu12        | 9.25.1.1  |
| nvidia-cufft-cu12        | 11.4.1.4  |

| 패키지                   | 버전         |
|-----------------------|------------|
| nvidia-curand-cu12    | 10.3.10.19 |
| nvidia-nvjitlink-cu12 | 12.9.86    |
| onnx                  | 1.22.0     |
| onnxruntime-gpu       | 1.23.2     |
| packaging             | 26.3       |
| pillow                | 12.3.0     |
| pluggy                | 1.6.0      |
| protobuf              | 7.36.1     |
| Pygments              | 2.21.0     |
| pytest                | 9.1.1      |
| scipy                 | 1.15.3     |
| sympy                 | 1.14.0     |
| tomli                 | 2.4.1      |
| typing_extensions     | 4.16.0     |

이 런타임이 강제하는 계약 상수: 파라미터 500 M(공유 텐서는 한 번만 계산), `encode` 호출당 1 s 이하, 행당 `ctx` 4 MB 이하, 아카이브 4 GB 이하. 제어 스텝 p95 40 ms 이하와 프레임당 생성 시간 p95 100 ms 이하는 이 런타임에서 측정합니다. 연산량(FLOPs)은 세어서 보고하되 상한은 없습니다.

원문(영문): 스타터 키트의 `runtime/RUNTIME.md`. 이 페이지는 그 한국어판이며, 값이 다르면 원문이 기준입니다.

## 채점 머신의 격리

제출물은 네트워크가 차단된 격리 환경에서, 각 문제가 문제 설명과 스타터 킷의 **RUNTIME.md**에 적은 메모리·시간 제한 안에서 실행됩니다 (문제 1은 에피소드당 120초와 에이전트 프로세스 메모리 6 GB, 문제 3은 채점 전에 측정하는 통과 조건).